

# AHTSYS

## A Semantic Framework for Autonomous Functional Control in Modular Embedded Hardware.

Joni Kemppainen (1,2), Jori-Aleksi Kemppainen (2)

(1) DEV Joni, Kukkulantie 4, 89400 Hyrynsalmi, Kainuu, Finland

(2) Alfrisin Oy, Tiilitörmäntie 6, 89400 Hyrynsalmi, Kainuu, Finland

Contact: [solutions@devjoni.com](mailto:solutions@devjoni.com)

6th of June 2026

### Abstract

Modular Printed Circuit Board (PCB) systems consist of functionally distinct modules that interface through standardized mechanical and electrical connectors. These systems offer significant advantages in rapid prototyping, industrial automation, and scientific instrumentation by allowing users to construct complex electronic assemblies from reusable, functional building blocks. While current modular ecosystems - ranging from widely adopted hobbyist platforms to industrial-grade interfaces - have successfully standardized the *transport layer* (utilizing protocols like SPI, I2C, or UART), a critical gap remains at the *control layer*. Existing solutions suffer from profound semantic fragmentation: even within a single manufacturer's ecosystem, each module typically requires unique integration code despite vast amounts of libraries or vendor specific SDKs. This lack of a unified semantic layer creates a high "integration tax," where the physical ease of modular connection is offset by the software complexity. AHTSYS addresses this fundamental limitation by introducing a vendor-independent control paradigm designed for the PCB level. AHTSYS is built upon two foundational components: (1) intelligent Functional Modules that self-describe their capabilities via a semantic On-Device Abstraction (OSA) Dictionary, and (2) a Host Controller that orchestrates the system through a Dynamic System Registry (DSR). This architecture is unified by a semantic key-value protocol that enables automated module discovery and standardized, intent-based control. By extending the principles of the PapaLink embedded device control protocol from the device level down to the individual PCB level, AHTSYS establishes a robust, modular framework for true plug-and-play embedded systems. This paradigm shifts the focus from connection-based compatibility to capability-based interoperability, significantly reducing integration overhead, enhancing cross-vendor portability, and providing a scalable foundation for the deployment and maintenance of complex, heterogeneous hardware environments. AHTSYS is proposed as a general-purpose architecture for embedded interoperability, remaining agnostic to specific transport-layer implementations.

**Keywords:** Communication protocol, modular PCB systems, automated module discovery, plug-and-play, embedded hardware, key-value abstraction, modular electronics, automation, robotics

# 1. Fragmentation Problem

Modular PCB systems exist in a diverse landscape of form factors: stackable modules, carrier-board-based card systems, wire-interconnected modules, industrial DIN-rail configurations and more. In this context, physical fragmentation - where modules from different manufacturers are not mechanically compatible - is an expected and often beneficial characteristic. It allows manufacturers to optimize their hardware for specific requirements, such as mechanical robustness, high computational performance, or cost-efficiency, enabling users to select the most appropriate tool for their application.

However, a more critical and detrimental form of fragmentation exists at the control layer. Even within a single manufacturer's ecosystem, modularity is often superficial. While most modern systems can electrically detect the presence of a module, this detection does not translate into functional readiness.

The current state-of-the-art integration workflow is characterized by a high "integration tax," following these repetitive steps:

1. **Physical Integration:** The module is connected to the system.
2. **Electrical Detection:** The presence of the module is identified via the bus.
3. **Library Discovery:** The user must manually identify the specific software library required for the module.
4. **Software Installation:** The appropriate drivers or libraries are integrated into the development environment.
5. **Implementation:** The user writes custom code to interface with the module's specific API.
6. **System Configuration:** The module is configured within the context of the broader system settings.
7. **Functional Integration:** The module's logic is orchestrated with the rest of the system.

Even when pre-written libraries are available, this process demands significant programming expertise, intensive system-specific configuration, and deep knowledge of the module's internal operational logic. As the number of modules in a system increases, the complexity of managing their interdependencies and conflicting requirements grows exponentially.

## 1.1. Limitations of Existing Paradigms

Historically, the industry has attempted to mitigate this through two primary models, both of which possess inherent flaws:

**The Open-Source Library Model:** In many open development ecosystems, users rely on a vast array of community-maintained libraries. While this avoids the need to write low-level drivers from scratch, it leads to **API Proliferation**. Each library introduces its own unique API, naming conventions, and architectural assumptions. The developer is forced to act as a "software glue," writing complex orchestration code to prevent libraries from clashing. In many cases, these libraries are fundamentally incompatible, competing for the same MCU resources, interrupts, or timing windows.

**The Proprietary SDK Model:** Common in scientific and industrial sectors, vendors provide highly optimized, closed Software Development Kits (SDKs). While these SDKs guarantee internal compatibility, they merely shift the burden of integration from the user to the vendor. For the end-user, this results in high maintenance costs and a heavy reliance on the vendor's release cycle. Adding new functionality becomes a slow, expensive process of managing and updating proprietary software stacks.

## 1.2. Technical Impact: Systemic Fragility and Complexity

The lack of a semantic layer leads to several technical inefficiencies:

- **Supply-Chain Fragility:** Modular systems currently behave as "glass cannons." If a module becomes obsolete or a manufacturer is forced to change a component (e.g., a different sensor IC), the underlying control logic often changes. This necessitates a complete rewrite of the integration code, making the entire system vulnerable to minor hardware variations.
- **Resource Bloat:** Because each module requires its own bespoke driver stack, the central controller (MCU) must possess significant memory and processing power to house multiple, often redundant, libraries. This leads to unnecessary hardware over-specification.
- **Observability Challenges:** Debugging, monitoring, and reverse-engineering become disproportionately complex when every module communicates through a different logical language.

## 1.3. Economic impact: Wasted resources

The technical shortcomings of current modular systems translate directly into economic burdens:

- **Engineering Overhead:** Excessive development time is diverted from high-value innovation toward the repetitive task of low-level software integration.
- **Technological Lock-in:** The high cost of learning and implementing a specific vendor's SDK creates a barrier to entry, effectively locking users into a single ecosystem and discouraging healthy competition.
- **Operational Inefficiency:** To support heavy SDKs and fragmented libraries, systems often require higher-specification MCUs, leading to increased Bill of Materials (BOM) costs, higher power consumption, and increased thermal output.
- **Sustainability and Waste:** The lack of semantic interoperability limits the lifecycle of modular hardware. While a passive adapter could physically bridge different connectors and transport standards (like SPI or I2C), the software barrier prevents the reuse of existing modules, contributing to electronic waste and preventing the creation of a truly circular modular economy.

## 1.4. The Need for a New Paradigm

Ultimately, these technical and economic inefficiencies represent a systemic barrier to true modularity. The industry has reached the limit of what transport-layer standardization can achieve. To unlock the full potential of modular hardware, a fundamental shift is required: moving away from hardware-centric integration and toward a semantic-centric architecture.

## 2. The Semantic Communication protocol

AHTSYS resolves the fragmentation problem by implementing a semantic abstraction layer directly at the hardware-firmware interface. By leveraging the principles of the PapaLink protocol, AHTSYS moves beyond mere electrical connectivity to achieve **functional interoperability**.

The framework is built upon a dual-layer architecture consisting of autonomous **Functional Modules** and a centralized **Host Controller**. This architecture ensures that the system's capabilities are defined by *what the hardware can do* (semantics) rather than *how it is electrically addressed*.

### 2.1. The Functional Module: Self-Describing Hardware

In the AHTSYS paradigm, a module is not a passive component; **it is a self-describing entity, powered by an onboard microcontroller (MCU)**. Every AHTSYS-compliant module has a dual-component firmware structure:

#### 2.1.1. The OSA Dictionary (Semantic Identity)

The core of the module's intelligence is the **On-Device Abstraction (OSA) Dictionary**. This is a structured collection of semantic key-value pairs that defines the module's capabilities.

- **Semantic Keys:** Instead of register addresses, the module exposes human-readable, standardized keys (e.g., {"temp": 22.5, "motor\_speed": 150, "status": 1}).
- **Standardized Semantics:** To ensure ecosystem-wide interoperability, AHTSYS utilizes the semantic definitions established by the PapaLink protocol. This ensures that a "light" key or a "rotation" key name carries the same meaning across different manufacturers.
- **Extensibility:** While the core semantics are standardized, the dictionary allows for vendor-specific extensions. Manufacturers can include their proprietary keys for advanced features without breaking the fundamental compatibility of the module.

#### 2.1.2. Local Control Logic (LCL)

The OSA Dictionary acts as the interface, but the **Local Control Logic** performs the work. This firmware component acts as the bridge between the abstract semantic layer and the physical hardware. When the Host Controller modifies a value in the OSA Dictionary, the Local Control Logic interprets that change and executes the corresponding physical action - such as adjusting a PWM signal for a motor, reading an ADC value from a sensor, or toggling a GPIO pin.

### 2.2. The Host Controller: The System Orchestrator

The Host Controller serves as the central intelligence of the AHTSYS ecosystem. Its primary role is to orchestrate the functional capabilities of the entire assembled system.

#### 2.2.1. Bus Management and Discovery

The Host Controller manages the underlying transport layer (the physical communication bus). It is responsible for continuous monitoring of the bus to detect the physical and electrical presence of new modules. Upon detection, the controller initiates a standardized handshake to identify the new hardware.

### 2.2.2. The Dynamic System Registry (DSR)

The defining feature of the Host Controller is the **Dynamic System Registry**. During the discovery phase, the controller queries each module for its OSA Dictionary. It then builds a real-time, centralized map of all available functional keys within the system.

This registry transforms a collection of disparate PCBs into a single, unified "functional pool." The controller knows exactly which keys are available, which modules they belong to, and what their current values are, without ever needing to know the low-level electrical specifics of the underlying modules.

## 2.3. The AHTSYS Operational Workflow

The power of AHTSYS is realized through its automated operational cycle, which enables true Plug-and-Play functionality:

1. **Automated Discovery:** A module is connected; the Host Controller detects its electrical presence via the transport bus.
2. **Semantic Handshake:** The Controller requests the module's OSA Dictionary.
3. **Registry Synchronization:** The Controller parses the dictionary and updates the Dynamic System Registry with the module's new capabilities.
4. **Intent-Based Control:** The user or application interacts with the system by requesting changes to semantic keys (e.g., `set_val("fan_speed", 80)`). The Controller routes this request to the correct module, which executes the command via its local control logic.

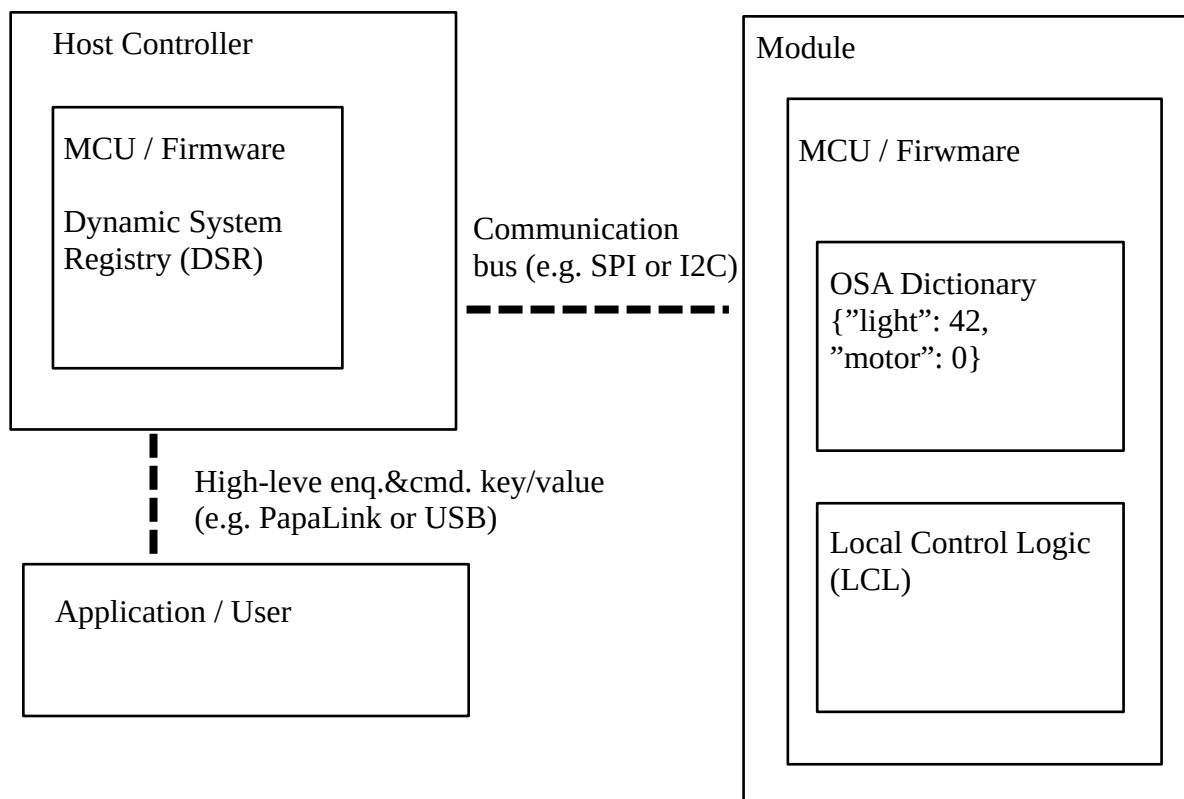
By decoupling the **intent of the command** from the **implementation of the action**, AHTSYS provides a scalable, robust, and vendor-independent foundation for modular electronic systems.

## 3. Conclusion

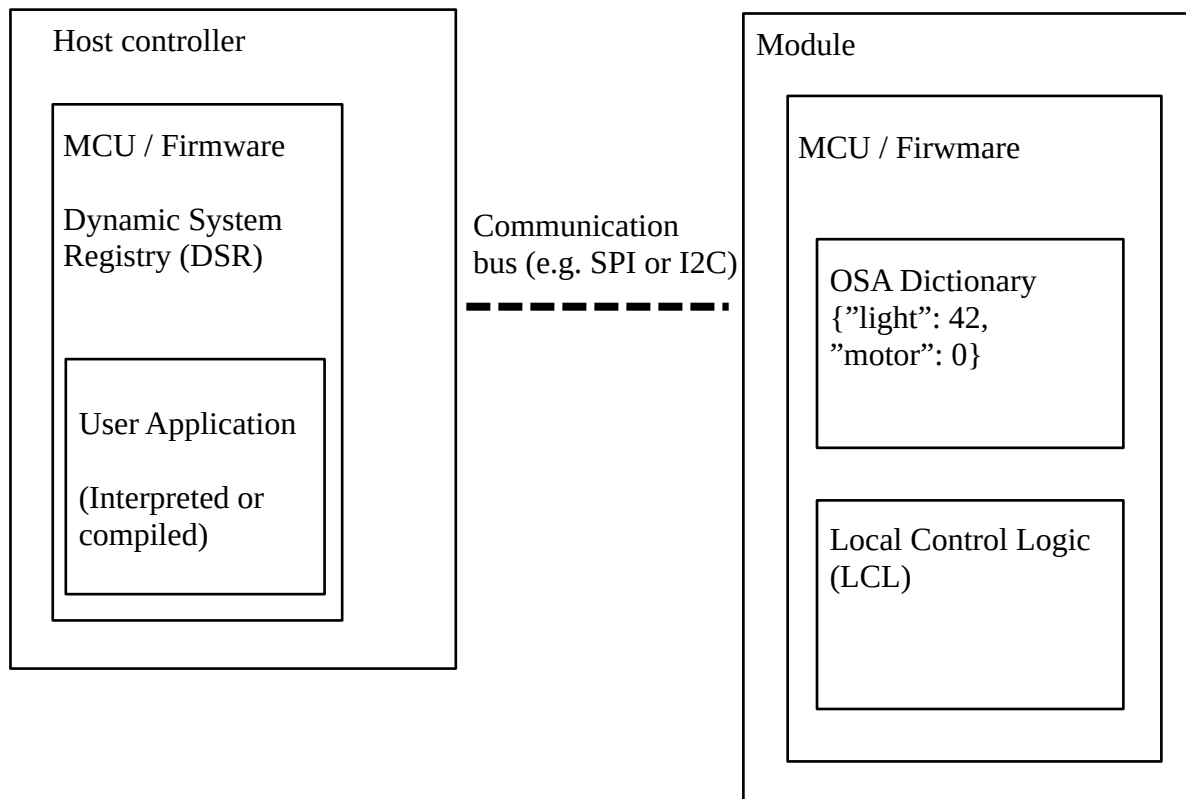
While physical and electrical standardization has advanced significantly, the lack of a semantic layer has left the industry trapped in a cycle of high integration costs, software fragmentation, and systemic fragility. The current paradigm - where connectivity is decoupled from functional readiness - imposes a heavy "integration tax" that stifles innovation and slows the deployment of complex systems.

AHTSYS offers a definitive departure from this fragmented landscape. By implementing the semantic principles of the PapaLink protocol at the PCB level, AHTSYS transforms modular hardware from a collection of disparate components into a unified, self-describing ecosystem. This shift from **connection-based compatibility** to **capability-based interoperability** provides a foundation for the next generation of embedded systems.

AHTSYS is more than a communication protocol; it is a fundamental architectural shift. By providing the semantic fabric required for autonomous functional discovery, AHTSYS paves the way for a future where modularity is truly plug-and-play, and where the complexity of hardware is seamlessly abstracted into the power of software.



**Drawing 1. System Architecture: External control scenario (e.g. computer control)**



**Drawing 2. System Architecture: Autonomous Scenario (e.g. edge computing)**

**Drawing 3. Functional Flowchart**

